Homogeneous variadic function parameters

Document number:	P1219R0
Date:	2018-10-08
Project:	Programming Language C++, Evolution Working Group

Reply-to:	James Touton <bekenn@gmail.com>

Table of Contents

Introduction

This paper seeks to expand the usefulness of variadic templates by allowing variadic function parameters to be declared using a single type. This would make the following construct legal:

```
template <class T>  // T is not a parameter pack...
void foo(T... vs);  // ...but vs is a parameter pack.
```

At first glance, this seems a simple and natural extension to variadic templates; after all, the following is already legal:

```
// Legal since C++11: T is not a parameter pack, but vs is.
template <class T, T... vs>
void foo();
```

In both examples, `vs` is a parameter pack, with each element having the type `T`. The meaning follows naturally from the rules for variadic templates, so it is a bit surprising that one is legal and the other is not. This paper explores the design space around making the first example legal.

Definitions

Throughout this paper, the term *homogeneous parameter pack* (or *homogeneous pack*) will be used to refer to a parameter pack containing values, all of the same type, where the pack's declaration does not expand a pack of types. A parameter pack containing values that are permitted to be of different types is referred to as a *heterogeneous parameter pack* (or *heterogeneous pack*). The term *homogeneous function parameter pack* refers to a homogeneous pack of function parameters, and the term *homogeneous template parameter pack* refers to a homogeneous pack of template parameters. The terms *heterogeneous function parameter pack* and *heterogeneous template parameter pack* refer to heterogeneous packs of function and template parameters, respectively.

Implementation Experience

An implementation of this proposal based on Clang can be found at https://github.com/Bekenn/clang/tree/func-parm-packs. It is believed to be very nearly both complete and correct, with only minor deviations in behavior from the wording provided in this paper. The remaining issues are expected to be resolved before the San Diego meeting.

Motivation and Scope

Meeting user expectations

This table showcases the current lack of symmetry between function parameter packs and function parameter packs:

	Template parameter pack	Function parameter pack

Heterogeneous	`// OK:`
	`template <class... TypePack, TypePack... vs>`
	`void f();`
		`// OK:`
		`template <class... TypePack>`
		`void f(TypePack... vs);`

Homogeneous	`// OK:`
	`template <class Type, Type... vs>`
	`void f();`
		`// Ill-formed:`
		`template <class Type>`
		`void f(Type... vs);`

The absence of homogeneous function parameter packs is a source of confusion among programmers. A cursory search of Stack Overflow turned up several questions ([1], [2], [3], [4], [5], [6], [7], [8]) that basically amount to asking how to write a function with a homogeneous parameter pack. Some work-arounds are suggested:

Recursion	`std::initializer_list`	Homogeneous pack (this proposal)

`template <class T>`
`T min(T v)`
`{`
`  return v;`
`}`
 	
`template <class T, class... Args>`
`T min(T v1, Args... vs)`
`{`
`  T v2 = min(vs...);`
`  return v2 < v1 ? v2 : v1;`
`}`
	`template <class T>`
	`T min(std::initializer_list<T> vs)`
	`  [[expects: vs.size() > 0]]`
	`{`
	`  auto i = vs.begin();`
	`  T v = *i++;`
	`  for (; i != vs.end(); ++i)`
	`  {`
	`  if (*i < v)`
	`  v = *i;`
	`  }`
	
	`  return v;`
	`}`
		`template <class T>`
		`T min(T v1, T... vs)`
		`{`
		`  return (v1, ..., (v1 = vs < v1 ? vs : v1));`
		`}`

The work-arounds suffer from a lack of clarity in the interface. The recursion approach advertises a function that can take variadic arguments of any type, but any attempt to call the function with a non-`T` will either cause the program to fail to compile, or lead to potentially unwanted implicit conversions. The `std::initializer_list` approach correctly advertises the accepted type, but is inflexible with mutability and requires the user to enclose its arguments in braces. In the `min` example above, the contract precondition could be removed if an explicit argument preceded the `std::initializer_list`, but the syntactic separation remains, resulting in calls such as `min(a, { b, c, d })`.

These patterns can also be found in the Library Fundamentals TS in the form of `make_array`, and even in the standard for `std::min` and `std::max`. All of these could arguably be written more naturally with homogeneous parameter packs.

Current	With homogeneous packs

`template <class T>`
`constexpr T min(initializer_list<T> t);`
 	
`template <class T, class Compare>`
`constexpr T min(initializer_list<T> t, Compare comp);`
	`template <class T>`
	`constexpr T min(T v1, T... vs);`
	
	`template <class Compare, class T>`
	`constexpr T min(Compare comp, T v1, T... vs);`

`template <class D = void, class... Types>`
`constexpr array<conditional_t<is_void_v<D>,`
`  common_type_t<Types...>, D>,`
`  sizeof...(Types)>`
`  make_array(Types&&... t);`
	`template <class T>`
	`constexpr auto make_array(T&&... t)`
	`  -> array<decay_t<T>, sizeof...(t)>;`

Compared to `std::initializer_list`

This table shows some of the differences between homogeneous packs and `std::initializer_list`.

	`std::initializer_list`	homogeneous packs
Can be used outside of a template?	Yes	No
Can mutate elements?	No	Yes
Can appear before other arguments?	Yes	No
Get the number of elements	`x.size()`	`sizeof...(x)`
Iterate over the elements	Range-based `for`	Fold expressions

Homogeneous function parameter packs are *not* intended as a replacement for `std::initializer_list`. Although there are certainly areas of overlap ([9]), each has its place. Whereas a `std::initializer_list` forms a range over its component elements and is passed as a single argument, the elements of a parameter pack are passed as distinct arguments. This can render a constructor taking a parameter pack uninvocable if another constructor is deemed a better match. Consider the case of container initialization:

```
template <class T>
class MyContainer
{
public:
    MyContainer(std::initializer_list<T> elems);
    // ...
};
```

The `std::initializer_list` constructor allows an instance of `MyContainer` to be constructed using list-initialization from a list of element values:

```
MyContainer<int> cont = { 5, 10 };  // invokes MyContainer<int>::MyContainer(std::initializer_list<int>)
```

Under the rules governing uniform initialization, this syntax continues to work with homogeneous parameter packs:

```
template <class T>
class MyContainer
{
public:
    template <> MyContainer(const T&... elems);
    // ...
};

MyContainer<int> cont = { 5, 10 };  // invokes MyContainer<int>::MyContainer(const int&, const int&)
```

...*unless* there is a competing constructor that is a better match:

```
template <class T>
class MyContainer
{
public:
    template <> MyContainer(const T&... elems);
    explicit MyContainer(const T& min, const T& max);
    // ...
};

MyContainer<int> cont = { 5, 10 };  // error: MyContainer<int>::MyContainer(const int&, const int&) is explicit
```

With the additional constructor, it is now impossible to invoke the constructor containing the homogeneous parameter pack when there are exactly two elements, regardless of the initialization syntax chosen. In contrast, the constructor with `std::initializer_list` is *always* chosen when using list-initialization, and can be unambiguously selected (or not) when using direct non-list initialization.

Declaration	With `std::initializer_list`	With homogeneous packs

	`template <class T>`
	`class MyContainer`
	`{`
	`public:`
	`  MyContainer(std::initializer_list<T> elems);`
	`  explicit MyContainer(const T& min, const T& max);`
	`  // ...`
	`};`
		`template <class T>`
		`class MyContainer`
		`{`
		`public:`
		`  template <> MyContainer(const T&... elems);`
		`  explicit MyContainer(const T& min, const T& max);`
		`  // ...`
		`};`

`MyContainer<int> cont{1, 2, 3};`	Selects `std::initializer_list` constructor	Selects homogeneous pack constructor
`MyContainer<int> cont = { 1, 2, 3 };`	Selects `std::initializer_list` constructor	Selects homogeneous pack constructor
`MyContainer<int> cont(1, 2, 3);`	Error: no matching constructor	Selects homogeneous pack constructor
`MyContainer<int> cont({ 1, 2, 3 });`	Selects `std::initializer_list` constructor	Error: no matching constructor
`MyContainer<int> cont{1, 2};`	Selects `std::initializer_list` constructor	Selects `explicit` constructor
`MyContainer<int> cont = { 1, 2 };`	Selects `std::initializer_list` constructor	Error: selects `explicit` constructor
`MyContainer<int> cont(1, 2);`	Selects `explicit` constructor	Selects `explicit` constructor
`MyContainer<int> cont({ 1, 2 });`	Selects `std::initializer_list` constructor	Error: no matching constructor

For this reason, homogeneous packs are likely inappropriate for constructors. Outside of constructors, where initializer lists must be separately enclosed in curly braces, homogeneous packs are likely a better option. One possible exception to this is when the API designer wishes to *move* or *forward* variadic constructor arguments rather than *copy* them. Frustratingly, the `std::initializer_list` template permits access to its elements only via references to `const`, whereas packs do not have this limitation.

Design Decisions

Declaring a homogeneous function parameter pack

A homogeneous function parameter pack is declared in exactly the same way as any other function parameter pack; the only difference is that the parameter declaration does not mention a template parameter pack:

```
template <class... T> void f(T... v);   // heterogeneous function parameter pack
template <class T>    void f(T... v);   // homogeneous function parameter pack
```

The size of a homogeneous function parameter pack is deduced from function arguments at the call site. This requires the pack to be placed in a deduced context, which means that a function can have at most one homogeneous function parameter pack, and the pack must appear at the end of the function parameter list. In all other respects, a homogeneous function parameter pack behaves no differently from any other parameter pack.

Lambdas

```
auto a = [](int... v) { return (1 * ... * v); };
```

The syntax for declaring a homogeneous function parameter pack in a lambda expression follows naturally from the syntax used for a function template. Just as packs can only be declared in templates, adding a homogeneous pack declaration to a lambda expression will cause it to become a generic lambda.

The template introducer

Function parameter packs can only appear in templates, and this proposal does nothing to change that. Under the current rules, function templates must be introduced using the `template` keyword, followed by the template parameter list in angle brackets. This requirement may change, depending on the outcome of ongoing discussions surrounding a terse syntax for concepts; this proposal is intended to work naturally with whatever outcome emerges.

Under the proposed rules, any function declaration that includes a parameter pack must be a template, and requires a *template-head*. If the parameter pack is the only part of the function declaration requiring it to be a template, and the pattern of the parameter pack does not contain any dependent names, then the template parameter list may be omitted. In this case, the angle brackets are still required. At first glance, this may appear to conflict with the syntax used for explicit specializations, but the compiler can disambiguate based on the presence of a parameter pack:

```
template <class T> void foo(T... v);        // #1 OK
template <> void foo(int... v);             // #2 OK, declares a template with an empty template parameter list
template <> void foo(int a);                // OK, declares an explicit specialization of #2
template <> void foo<>(int a, int b);       // OK, declares an explicit specialization of #2
template <> void foo(float a);              // OK, declares an explicit specialization of #1
template <> void foo<int>(int a, int b);    // OK, declares an explicit specialization of #1
void bar(int... v);                         // Error: parameter pack without template
template <> void baz();                     // Error: no template parameters or packs, and does not specialize an existing template
```

Alternative: Allow the angle brackets to be omitted when no template parameters are needed.

```
template void foo(int... v);
```

This removes the apparent conflict with explicit specialization syntax, but introduces the same apparent conflict with the syntax for explicit instantiations. In both cases, there is no actual conflict; the compiler can tell that the function is a template by the presence of the parameter pack. This approach was considered and rejected because it introduces an irregularity into the grammar. Currently, all templates are introduced with a parameter list enclosed in angle brackets; removing them would be akin to removing the parentheses on a parameterless function declaration. The only declaration that uses the `template` keyword without angle brackets is the explicit instantiation declaration, and those do not declare templates.

Alternative: Allow the entire *template-head* to be omitted when no template parameters are needed.

```
void foo(int... v);
```

This idea is certainly attractive, but it has already been discussed extensively in relation to a terse syntax for concepts, and this paper has nothing new to add to that discussion. All of the concerns brought up when discussing concepts are equally applicable here. If a terse syntax for concepts can be agreed upon, homogeneous function parameter packs will likely fit into that syntax without any problems.

Alternative: Allow or require a template parameter for the size of the pack.

```
template <size_t Len> void foo(int...[Len] v);
```

This would allow the size of the homogeneous pack to be explicitly specified, which could be useful in some situations. Making the length of the pack a parameter would also make clear in the syntax that the length is an axis of specialization; under the currently proposed rules, this is still true, but the fact is hidden from the user, and the length can only be deduced rather than specified.

The drawback to this approach is a matter of regularity. While the language allows the user to query the size of a pack using the `sizeof...` operator, it does not currently allow the size to be explicitly specified. If this facility were added, it could reasonably be applied to homogeneous template parameter packs as well as homogeneous function paremeter packs. It would make little sense to try to apply this to heterogeneous packs, since the size must match the number of arguments passed to the template parameter pack. Since this facility would necessarily be optional for homogeneous template parameter packs, consistency demands that it should also be optional for homogeneous function parameter packs.

Alternative: Allow empty angle brackets even when there is no homogeneous parameter pack.

```
template <> void foo(int n);
```

The proposed rules the *template-parameter-list* optional in a *template-head* in order to permit homogeneous packs with non-dependent types. The proposed rules also add semantic constraints requiring at least one template parameter whenever there is no homogeneous pack. The semantic constraints could be relaxed, allowing trivial templates that only permit vacuous specialization. This would not be very useful; these trivial templates would behave like normal non-template entities in almost every way imaginable, aside from syntactic minutiae (for instance, a trivial function template would be implicitly inline).

Apart from minor simplifications in the language specification, about the only "benefit" of this approach would be to make well-formed the token sequence `[]<>(){}` for the amusement of language nerds.

The Oxford variadic comma

In the C programming language, the appearance of an ellipsis in the *parameter-type-list* of a function declarator indicates that the function accepts a variable number of arguments of varying types following the last formal parameter in the list. Such an ellipsis will henceforth be referred to as a *varargs ellipsis* to distinguish it from the ellipsis used in the declaration of a parameter pack. To be syntactically valid in C, a varargs ellipsis must be preceded by at least one parameter declaration and an intervening comma:

> *parameter-type-list*:
> *parameter-list*
> *parameter-list* `,` `...`

C++ inherits this behavior, but expands the syntax, no longer requiring either the preceding parameter declaration or the intervening comma:

> *parameter-declaration-clause*:
> *parameter-declaration-list*$_{opt}$ `...`$_{opt}$
> *parameter-declaration-list* `,` `...`

When paired with function parameter packs, this creates a syntactic ambiguity that is currently resolved via a disambiguation rule: When an ellipsis that appears in a function parameter list might be part of an abstract (nameless) declarator, it is treated as such if the parameter's type names an unexpanded parameter pack or contains `auto`; otherwise, it is a varargs ellipsis. At present, this rule effectively disambiguates in favor of a parameter pack whenever doing so produces a well-formed result.

Example (status quo):

```
template <class... T>
void f(T...); // declares a function with a variadic parameter pack

template <class T>
void f(T...); // same as void f(T, ...)
```

With homogeneous function parameter packs, this disambiguation rule needs to be revisited. It would be very natural to interpret the second declaration above as a function template with a homogeneous parameter pack, and that is the resolution proposed here. By requiring a comma between a parameter list and a varargs ellipsis, the disambiguation rule can be dropped entirely, simplifying the language without losing any functionality or degrading compatibility with C.

This is a breaking change, but likely not a very impactful one. In the parlance of P0684R2, this would be a "Very Good Change": Compilers can issue warnings in C++17 mode (or earlier) whenever the disambiguation rule is resolved in favor of a varargs ellipsis, and the warning can be resolved without any change in meaning by the addition of a single character. Moreover, the number of instances should be vanishingly small. In modern C++ code, the varargs ellipsis has largely been superseded by function parameter packs. Today, apart from SFINAE uses where the disambiguation rule doesn't apply, the varargs ellipsis can mainly be found in header files that are intended to be consumed by both C and C++ code, in order to facilitate interoperability between the two languages. In these cases, the declarations must conform to the rules imposed by C syntax, and so they will already conform to the rules proposed here. Lastly, personal experience suggests that the vast majority of C++ users aren't even aware that the comma preceding a varargs ellipsis is optional.

Wording

All modifications are presented relative to N4762. "*[...]*" indicates elided content that is to remain unchanged.

Modify the grammar synopsis in §7.5.5 [expr.prim.lambda]:

> *lambda-expression*:
> *lambda-introducer compound-statement*
> *lambda-introducer lambda-declarator requires-clause*$_{opt}$ *compound-statement*
> *lambda-introducer* < *template-parameter-list*$_{opt}$ > *requires-clause*$_{opt}$ *compound-statement*
> *lambda-introducer* < *template-parameter-list*$_{opt}$ > *requires-clause*$_{opt}$ *lambda-declarator requires-clause*$_{opt}$ *compound-statement*
>
> *[...]*

Modify §7.5.5 [expr.prim.lambda] paragraph 5:

> A lambda is a generic lambda if ~~the `auto` *type-specifier* appears as one of the *decl-specifiers* in the *decl-specifier-seq* of a *parameter-declaration* of the *lambda-expression*, or if the lambda has a *template-parameter-list*.~~
>
> — the `auto` *type-specifier* appears as one of the *decl-specifiers* in the *decl-specifier-seq* of a *parameter-declaration* of the *lambda-expression*,
>
> — the lambda has a *template-parameter-list*, or
>
> — the lambda has a *lambda-declarator* and any *parameter-declaration* declares a parameter pack ([temp.variadic]).
>
> [*Example:*
>
> ```
> int i = [](int i, auto a) { return i; }(3, 4);          // OK: a generic lambda
> int j = []<class T>(T t, int i) { return i; }(3, 4);    // OK: a generic lambda
> int k = [](int... i) { return (0 + ... + i); }(3, 4);   // OK: a generic lambda
> ```
>
> — *end example*]

Modify §9.2.3.5 [dcl.fct] paragraph 3:

> A type of either form is a function type.
>
> *parameter-declaration-clause*:
> *parameter-declaration-list*$_{opt}$ ~~`,`$_{opt}$~~
> `...`
> *parameter-declaration-list* `,` `...`
>
> *[...]*

Modify §9.2.3.5 [dcl.fct] paragraph 4:

> *[...]* If the *parameter-declaration-clause* terminates with an ellipsis or a function parameter pack ([temp.variadic]), the number of arguments shall be equal to or greater than the number of parameters that do not have a default argument and are not function parameter packs. ~~Where~~

~~syntactically correct and where "`, ...`" is not part of an *abstract-declarator*, "`, ...`" is synonymous with "`...`".~~ [*Example:* The declaration

```
int printf(const char*, ...);
```

declares a function that can be called with varying numbers and types of arguments.

```
printf("hello world");
printf("a=%d b=%d", a, b);
```

However, the first argument must be of a type that can be converted to a const char* — *end example*] [*Note:* The standard header `<cstdarg>` contains a mechanism for accessing arguments passed using the ellipsis (see [expr.call] and [support.runtime]). — *end note*]

Modify §9.2.3.5 [dcl.fct] paragraph 17:

A *declarator-id* or *abstract-declarator* containing an ellipsis shall only be used in a *parameter-declaration*. When it is part of a *parameter-declaration-clause*, the *parameter-declaration* declares a function parameter pack ([temp.variadic]). Otherwise, the *parameter-declaration* is part of a *template-parameter-list* and declares a template parameter pack; see [temp.param]. A function parameter pack is a pack expansion ([temp.variadic]) when the type of the parameter contains one or more template parameter packs that have not otherwise been expanded. [*Example:*

```
template<typename... T> void f(T (* ...t)(int, int));

int add(int, int);
float subtract(int, int);

void g() {
  f(add, subtract);
}
```

— *end example*]

Delete §9.2.3.5 [dcl.fct] paragraph 18 and the accompanying footnote:

~~There is a syntactic ambiguity when an ellipsis occurs at the end of a *parameter-declaration-clause* without a preceding comma. In this case, the ellipsis is parsed as part of the *abstract-declarator* if the type of the parameter either names a template parameter pack that has not been expanded or contains `auto`; otherwise, it is parsed as part of the *parameter-declaration-clause*.~~

Modify §11.4 [over.over] paragraph 2:

If the name is a function template, template argument deduction is done ([temp.deduct.funcaddr]), and if the argument deduction succeeds, ~~the resulting template argument list is used to generate~~ a single function template specialization~~, which~~ is generated using the resulting template argument list and the deduced number of elements for the trailing homogeneous function parameter pack (if present). The generated function template specialization is then added to the set of overloaded functions considered. [*Note:* As described in [temp.arg.explicit], if deduction fails and the function template name is followed by an explicit template argument list, the *template-id* is then examined to see whether it identifies a single function template specialization. If it does, the *template-id* is considered to be an lvalue for that function template specialization. The target type is not used in that determination. — *end note*]

Modify §12 [temp] paragraph 1:

A *template* defines a family of classes, functions, or variables, an alias for a family of types, or a concept.

template-declaration:
    template-head declaration
    template-head concept-definition

template-head:
    template < *template-parameter-list*$_{opt}$ > *requires-clause*$_{opt}$

template-parameter-list:
    template-parameter
    template-parameter-list , *template-parameter*

[...]

Modify §12.3 [temp.arg] paragraph 4:

When a template declares no *template-paremeters*, or when template argument packs or default *template-argument*s are used, a *template-argument* list can be empty. In that case the empty <> brackets shall still be used as the *template-argument-list*. [*Example:*

```
template<class T = char> class String;
String<>* p;                    // OK: String<char>
String* q;                      // syntax error
```

```
template<class ... Elements> class Tuple;
Tuple<>* t;                    // OK: Elements is empty
Tuple* u;                      // syntax error
```
— *end example*]

Delete §12.6 [temp.decls] paragraph 3 (redundant, moved to [temp.alias]):

~~Because an *alias-declaration* cannot declare a *template-id*, it is not possible to partially or explicitly specialize an alias template.~~

Insert a new paragraph after §12.6 [temp.decls] paragraph 2 (pending a resolution to CWG issue 1711):

The declaration of a primary template for a variable or static data member shall declare at least one *template-parameter*.

Modify §12.6.3 [temp.variadic] paragraph 5:

A pack expansion consists of a pattern and an ellipsis, the instantiation of which produces zero or more instantiations of the pattern in a list (described below). The form of the pattern depends on the context in which the expansion occurs. Pack expansions can occur in the following contexts:

— In a function parameter pack that is a pack expansion ([dcl.fct]); the pattern is the *parameter-declaration* without the ellipsis.

[...]

Insert a new paragraph after §12.6.3 [temp.variadic] paragraph 5:

A function parameter pack whose declaration is not a pack expansion is a *homogeneous function parameter pack*. A homogeneous function parameter pack shall only appear at the end of the *parameter-declaration-clause* of a function template or a generic lambda. [*Note:* Because a homogeneous function parameter pack has no corresponding template parameter packs, the length of the pack must be deduced at the point of use; therefore, the pack cannot appear in a non-deduced context. — *end note*]

Modify §12.6.3 [temp.variadic] paragraph 10:

[...] [*Example:*

```
template<~~typename ...Args~~>
bool all(~~Args~~bool ...args) { return (... && args); }
bool b = all(true, true, true, false);
```

Within the instantiation of `all`, the returned expression expands to `((true && true) && true) && false`, which evaluates to `false`. — *end example*] *[...]*

Modify §12.6.5 [temp.class.spec] paragraph 1:

A *primary class template* declaration is one in which the class template name is an identifier. A template declaration in which the class template name is a *simple-template-id* is a *partial specialization* of the class template named in the *simple-template-id*. A partial specialization of a class template provides an alternative definition of the template that is used instead of the primary definition when the arguments in a specialization match those given in the partial specialization ([temp.class.spec.match]). The primary template shall be declared before any specializations of that template and shall declare at least one *template-parameter*. A partial specialization shall be declared before the first use of a class template specialization that would make use of the partial specialization as the result of an implicit or explicit instantiation in every translation unit in which such a use occurs; no diagnostic is required.

Insert a new paragraph after §12.6.6 [temp.fct] paragraph 2:

A function template declaration shall either declare a homogeneous function paraemeter pack or declare at least one *template-parameter*.

Modify §12.6.7 [temp.alias] paragraph 1:

A *template-declaration* in which the declaration is an *alias-declaration* ([dcl.dcl]) declares the identifier to be an *alias template*. An alias template is a name for a family of types. The name of the alias template is a *template-name*. [*Note:* Because an *alias-declaration* cannot declare a *template-id*, it is not possible to partially or explicitly specialize an alias template. — *end note*]

Insert a new paragraph after §12.6.7 [temp.alias] paragraph 1:

An alias template declaration shall declare at least one *template-parameter*.

Modify §12.6.8 [temp.concept] paragraph 3:

A *concept-definition* shall appear at namespace scope ([basic.scope.namespace]) and shall declare at least one *template-parameter*.

Modify §12.7.2 [temp.dep] paragraph 1:

[...] An expression may be *type-dependent* (that is, its type may depend on a template parameter or the number of elements in a parameter pack) or *value-dependent* (that is, its value when evaluated as a constant expression ([expr.const]) may depend on a template parameter) as described in this subclause. *[...]*

Modify §12.7.2.1 [temp.dep.type] paragraph 9:

A type is dependent if it is

- a template parameter,
- a member of an unknown specialization,
- a nested class or enumeration that is a dependent member of the current instantiation,
- a cv-qualified type where the cv-unqualified type is dependent,
- a compound type constructed from any dependent type,
- an array type whose element type is dependent or whose bound (if any) is value-dependent,
- a function type whose parameter-type-list contains a parameter pack,
- a function type whose exception specification is value-dependent,
- denoted by a *simple-template-id* in which either the template name is a template parameter or any of the template arguments is a dependent type or an expression that is type-dependent or value-dependent or is a pack expansion [*Note:* This includes an *injected-class-name* ([class]) of a class template used without a *template-argument-list*. — *end note*] , or
- denoted by `decltype(`*expression*`)`, where *expression* is type-dependent ([temp.dep.expr]).

Modify §12.9.2 [temp.deduct] paragraph 1:

When a function template specialization is referenced, all of the template arguments shall have values and the number of elements in each function parameter pack shall be known. The ~~values~~template arguments can be explicitly specified or, in some cases, be deduced from the use or obtained from default *template-argument*s. The number of elements in a homogeneous function parameter pack must be deduced. [*Note:* If a function template declaration includes a homogeneous function parameter pack, then a *template-id* is never sufficient to refer to an individual specialization of the template. — *end note*] [*Example:*

```
template<class T> class Array { /* ... */ };
template<class T> void sort(Array<T>& v);

void f(Array<dcomplex>& cv, Array<int>& ci) {
  sort(cv);                   // calls sort(Array<dcomplex>&)
  sort(ci);                   // calls sort(Array<int>&)
}
```

~~and~~

```
template<class U, class V> U convert(V v);

void g(double d) {
  int i = convert<int>(d);    // calls convert<int,double>(double)
  int c = convert<char>(d);   // calls convert<char,double>(double)
}

template<> int sum(int... n) { return (0 + ... + n); }

void h(int x, int y, int z) {
  int i = sum(x, y, z);       // calls sum<>(int,int,int)
}
```

— *end example*]

Modify §12.9.2 [temp.deduct] paragraph 5:

[...] When all template arguments have been deduced or obtained from default template arguments, all uses of template parameters in the template parameter list of the template and the function type are replaced with the corresponding deduced or default argument values. If the function type has a homogeneous function parameter pack and the number of elements for the pack has been deduced, the pack is replaced with

a corresponding number of function parameters, each an instance of the pack's pattern. If the substitution results in an invalid type, as described above, type deduction fails. If the function template has associated constraints ([temp.constr.decl]), those constraints are checked for satisfaction ([temp.constr.constr]). If the constraints are not satisfied, type deduction fails.

Modify §12.9.2 [temp.deduct] paragraph 11:

[*Note:* Type deduction may fail for the following reasons: *[...]*

— Attempting to deduce the type of a function template specialization from a *template-id* when the function template contains a homogeneous function parameter pack. [*Example:*

```
template <class T> void f(T... v);
auto p = &f<int>;   // ambiguous; function parameter pack of unknown size
```

— end example]

— end note]

Modify §12.9.2.1 [temp.deduct.call] paragraph 1:

[...] For a function parameter pack that occurs at the end of the *parameter-declaration-list*, the number of elements in the pack is determined as the number of arguments remaining in the call. ~~d~~Deduction is performed for each remaining argument ~~of the call~~, taking the type `P` of the *declarator-id* of the function parameter pack as the corresponding function template parameter type. Each deduction deduces template arguments for subsequent positions in the template parameter packs expanded by the function parameter pack (if any). When a function parameter pack appears in a non-deduced context ([temp.deduct.type]), the type of that pack is never deduced. [*Example:*

```
template<class ... Types> void f(Types& ...);
template<class T1, class ... Types> void g(T1, Types ...);
template<class T1, class ... Types> void g1(Types ..., T1);
void h(int x, float& y) {
  const int z = x;
  f(x, y, z);                   // Types is deduced to int, float, const int
  g(x, y, z);                   // T1 is deduced to int; Types is deduced to float, int
  g1(x, y, z);                  // error: Types is not deduced
  g1<int, int, int>(x, y, z);   // OK, no deduction occurs
}
```

— end example]

Modify §12.9.2.4 [temp.deduct.partial] paragraph 8:

Using the resulting types `P` and `A`, the deduction is then done as described in [temp.deduct.type]. If `P` is a function parameter pack, the type `A` of each remaining parameter type of the argument template is compared with the type `P` of the *declarator-id* of the function parameter pack. Each comparison deduces template arguments for subsequent positions in the template parameter packs expanded by the function parameter pack (if any). *[...]*

Modify §12.9.2.5 [temp.deduct.type] paragraph 10:

[...] If the *parameter-declaration* corresponding to $\texttt{P}_i$ is a function parameter pack, then the type of its *declarator-id* is compared with each remaining parameter type in the *parameter-type-list* of `A`. Each comparison deduces template arguments for subsequent positions in the template parameter packs expanded by the function parameter pack (if any). *[...]*

Insert a new paragraph after §12.10 [temp.deduct.guide] paragraph 3:

A *template-declaration* in which the declaration is a *deduction-guide* shall declare at least one *template-parameter*.

References

1. Specifying one type for all arguments passed to variadic function or variadic template function w/out using array, vector, structs, etc? (https://stackoverflow.com/questions/3703658/)
2. C++ parameter pack, constrained to have instances of a single type? (https://stackoverflow.com/questions/38528801/)
3. Variadic template parameters of one specific type (https://stackoverflow.com/questions/30773216/)
4. C++ parameter pack with single type enforced in arguments (https://stackoverflow.com/questions/47470874/)
5. C++11 variable number of arguments, same specific type (https://stackoverflow.com/questions/18017543/)
6. Enforce variadic template of certain type (https://stackoverflow.com/questions/30346652/)

7. variadic template of a specific type
(https://stackoverflow.com/questions/13636290/)
8. C++11: Variadic Homogeneous Non-POD Template Function Arguments?
(https://stackoverflow.com/questions/9762531/)
9. Bjarne Stroustrup, Variadic Templates
(https://parasol.tamu.edu/people/bs/622-GP/variadic-templates-and-tuples.pdf)